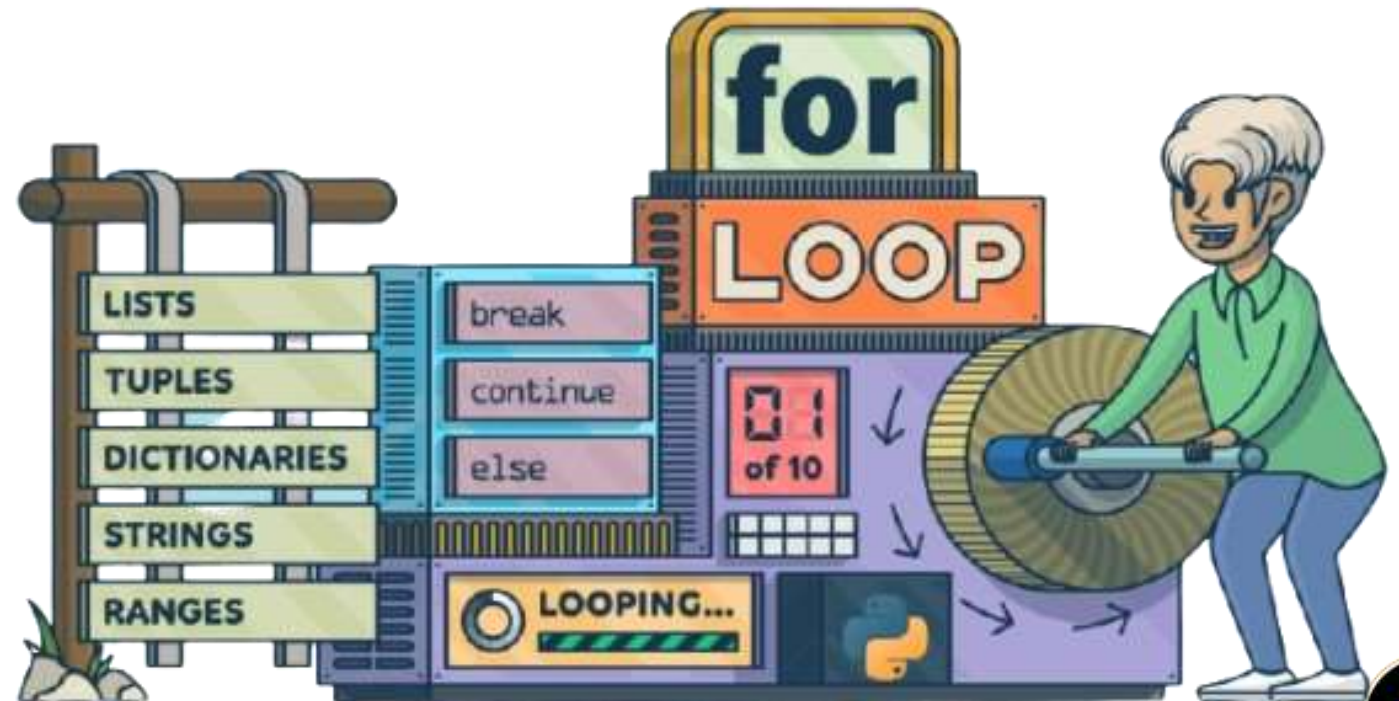
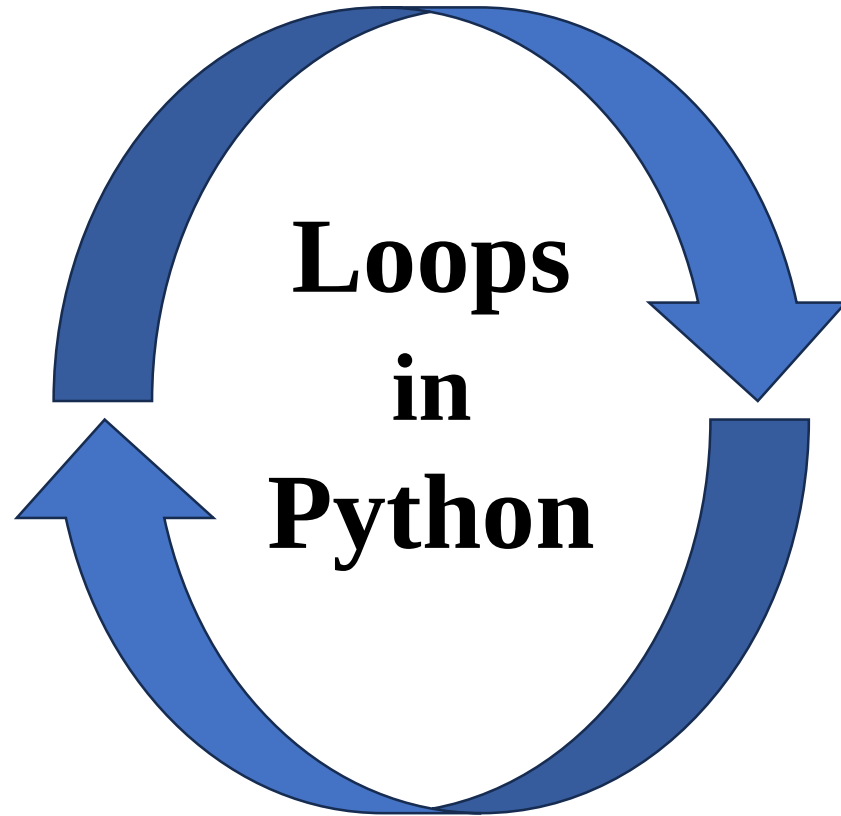




Loops in Python

In programming, a **loop** is a structure that allows us to **repeat a block of code** multiple times without writing it over and over. This is extremely useful when we want to perform repetitive tasks like:

- Displaying a message multiple times ➡
- Traversing elements of a list or string ↻
- Running until a certain condition is met ✓

Python provides two main looping structures:

- **for loop** – used when the number of iterations is known
- **while loop** – used when the number of iterations is unknown

💡 Problem 1: Print "Python is Fun!" 5 times

```
for i in range(5):  
    print("Python is Fun!")
```



While Loop in Python

- A while loop in Python **keeps running** as long as the **condition evaluates to True**.
- It is suitable when the number of repetitions is **not known in advance**.

Syntax:

```
while condition:  
    # block of code
```

Key Points:

- The **condition is checked** before every iteration.
- If the condition is False, the loop terminates.
- Inside the loop, you must **change a variable** to eventually stop the loop (else → infinite loop).

Infinite Loop Risk:

If you forget to update the condition variable, the loop will never stop!





💡 **Problem 2: Print numbers 1 to 5 using while loop**

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

💡 **Problem 3: Print even numbers from 2 to 10**

```
i = 2
while i <= 10:
    print(i)
    i += 2
```

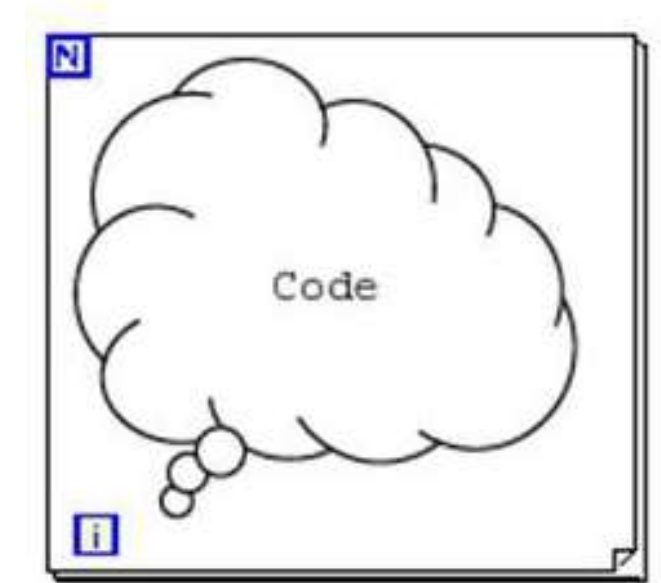


➤ For Loop in Python

- A for loop is used to iterate over a **sequence** (like list, string, tuple, range, etc.).
- Unlike many other languages, Python's for is more like "**for each**".
- It **automatically handles** the start, stop, and step.

Syntax:

```
for variable in sequence:  
    # block of code
```



✈ How it Works:

- It picks each element from the sequence, assigns it to the loop variable, and executes the code block.





💡 Problem 4: Print characters of a string

```
text = "Hello"  
for ch in text:  
    print(ch)
```

💡 Problem 5: Print numbers from 1 to 10

```
for i in range(1, 11):  
    print(i)
```



➤ range() Function

The range() function is used to generate a sequence of numbers. It's most commonly used with a for loop.

Syntax:

```
range(start, stop, step)
```

- **start** → beginning number (optional, default is 0)
- **stop** → end number (exclusive)
- **step** → difference between numbers (optional, default is 1)

Examples:

```
range(5) → 0, 1, 2, 3, 4
```

```
range(2, 6) → 2, 3, 4, 5
```

```
range(10, 0, -2) → 10, 8, 6, 4, 2
```





💡 **Problem 6: Print odd numbers from 1 to 19**

```
for i in range(1, 20, 2):  
    print(i)
```

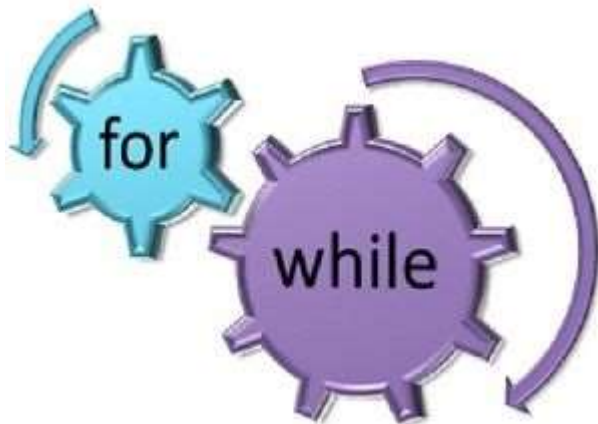
💡 **Problem 7: Print numbers in reverse from 10 to 1**

```
for i in range(10, 0, -1):  
    print(i)
```



➤ for vs while Loop – When to Use What?

Feature	for loop	while loop
Iteration Type	Definite (known number of times)	Indefinite (until condition is False)
Best For	Traversing sequences	Condition-based repetition
Infinite Loop Risk	Low	High if condition is not updated properly
Syntax	Shorter with range()	More flexible



Nested Loops

A **nested loop** means placing one loop inside another. The **inner loop runs completely** for **each iteration** of the outer loop.

Useful for:

- **Pattern printing** (stars, numbers)
- **Matrix traversal**
- **Solving 2D problems**

 **Structure:**

```
for i in range(outer):  
    for j in range(inner):  
        # inner block
```

 **Example:**

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```





💡 Problem 8: Print star pattern

```
rows = 5
for i in range(1, rows+1):
    for j in range(i):
        print("*", end=" ")
    print()
```

💡 Problem 9: Multiplication table using nested loop

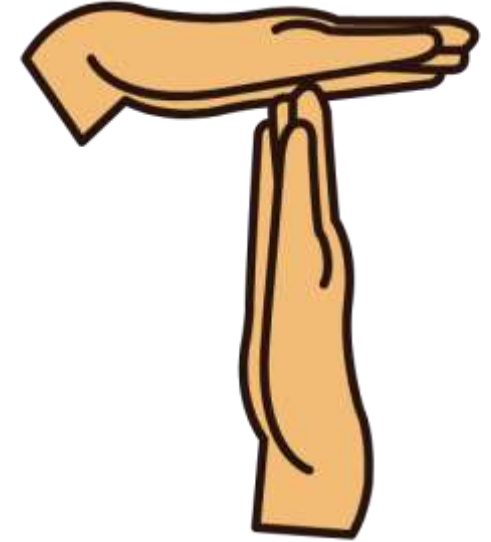
```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()
```



➤ Loop Control Statements

Python provides special keywords to **control the flow of loops**:

- a) **break**
 - Terminates the loop **immediately**, regardless of the condition.
- ↻ b) **continue**
 - Skips the **current iteration** and jumps to the **next** one.
- c) **pass**
 - **Does nothing**; used as a placeholder when syntax requires a block but logic is pending.





💡 Problem 10: Stop loop when number is 5

```
for i in range(1, 11):  
    if i == 5:  
        break  
    print(i)
```

💡 Problem 11: Skip number 3

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```





💡 Problem 12: Use pass to skip logic temporarily

```
for i in range(3):  
    pass # logic will be added later
```

💡 Problem 13: Sum of N Numbers

```
n = int(input("Enter a number: "))  
sum = 0  
for i in range(1, n+1):  
    sum += i  
print("Sum =", sum)
```





💡 Problem 14: Factorial of a Number

```
n = int(input("Enter a number: "))  
fact = 1  
for i in range(1, n+1):  
    fact *= i  
print("Factorial:", fact)
```

💡 Problem 15: Count the digits of a number

```
n = int(input("Enter a number: "))  
count = 0  
while n > 0:  
    n = n // 10  
    count += 1  
print("Number of digits:", count)
```



➤ ! Problem 16: Prime Number Check

```
n = int(input("Enter a number: "))
is_prime = True

if n < 2:
    is_prime = False
else:
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            is_prime = False
            break

if is_prime:
    print("Prime Number")
else:
    print("Not a Prime Number")
```





💡 **Problem 17: Print all prime numbers from 1 to 50**

```
for num in range(2, 51):  
    is_prime = True  
    for i in range(2, int(num**0.5)+1):  
        if num % i == 0:  
            is_prime = False  
            break  
    if is_prime:  
        print(num, end=" ")
```



